

User-defined data types



User-defined data type

Structure definition



Structure



- Structure is a user-defined data type
- Creates a data type that can be used to group items of possibly different types into a single type
- On the other hand, an array is a homogenous collection. All elements have the same type.



Syntax of structure

- **struct** keyword is used to create a structure

```
struct structure_name {  
    data_type member1;  
    data_type member2;  
    // ... more members  
};
```

Declaring/Defining structures

- The following declaration creates a user-defined data type struct student

```
struct student {  
    char name[30];  
    int roll_no;  
};
```

- Define the struct data type in the same way like the basic data types

struct student	stud1, stud2;
	
Data type	Variables

Both Declare and Define

- Structures can be both declared and defined together creating a derived data type
 - The following examples create two variables of derived data type **struct student**

```
struct student {  
    char name[30];  
    int roll_no;  
}stud1, stud2;
```

This creates a new "template" called struct student. The problem is that every time you want to declare a variable of this type, you have to write struct student (you can't just write student):

Using typedef with structures

Syntax

```
typedef struct {  
    type member 1,  
    ....  
} type_name;
```

Example

```
typedef struct {  
    char name[30];  
    int roll_no;  
} student;  
student stud1, stud2;
```

typedef (or "type definition") is a keyword that lets you create an alias or a new, simpler name for an existing data type.

For example, you could do this: typedef unsigned long ulong;

Now, instead of writing unsigned long my_num;, you can just write ulong my_num;.

*It means stud1 is a variable that contains its own name and roll_no.
stud2 is a separate variable that also contains its own name and roll_no.*

Initializing structures

- Structure members cannot be initialized inside the structure definition, why?

```
struct student {  
    char name[30] = "Alice";  
    int roll_no = 100;  
};
```

- This will result in a compiler error,
- Structure members can be initialized using curly braces during declaration.
`struct student stud1={"Alice",100};`

Accessing structure members

- dot operator (.) is called the member access operator.
- Syntax is
 <structure variable>.<member name>
- Example

```
struct student {  
    char name[30];  
    int roll_no;  
};
```

```
struct student stud1={"Alice",100};  
printf("Name: %s\n",stud1.name);  
Printf("Roll no.%d\n",stud1.roll no);
```

struct syntax

■ Declaration of a structure: 2 ways

- **Option 1:**

```
struct <struct_name>{
    <type> <identifier_list>;
    <type> <identifier_list>;
    ...
};
```

- **Option 2:**

```
typedef struct {
    <type> <identifier_list>;
    <type> <identifier_list>;
    ...
} <struct_name>;
```

■ Defining structure variables: 3 ways

```
struct <struct_name>{
    <type> <identifier_list>;
    <type> <identifier_list>;
    ...
} s1, s2 ; // Both declaration and definition together
```

- If structure declared as **option 1**
`struct <struct_name> s1, s2;`

- If structure declared as **option 2**
`<struct_name> s1, s2;`

Ex:



- Write a program to store the following information about a student.
 - Name, roll number, course no, course name and mark



Array of structures

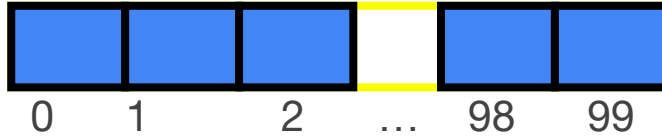
- Defining an array of structures is same as defining an array of fundamental data types

```
struct student {  
    char name[30];  
    int roll_no;  
    float marks;  
};  
  
struct student stud[42];
```

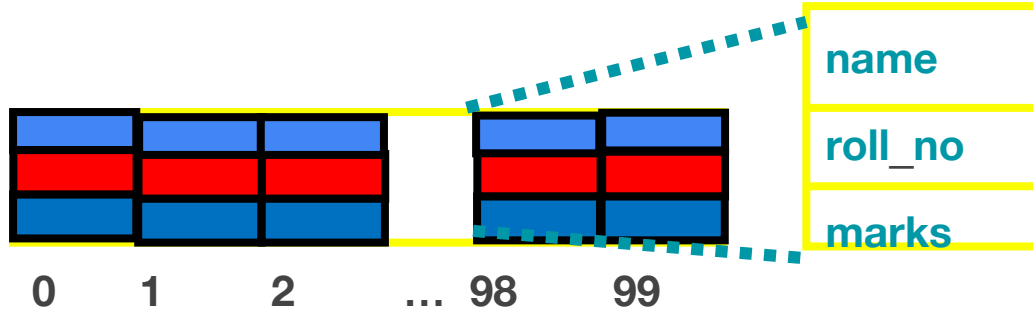
<u>stud[0]</u>	stud[0].name	stud[0].roll_no	stud[0].marks
<u>stud[1]</u>	stud[1].name	stud[1].roll_no	stud[1].marks
.....			

Array of structures

- An ordinary array: One type of data



- An array of structs: Multiple types of data in each array element.





User-defined data type

Nested structures



Nested structures



- Nested structure: means structure within a structure

```
struct struct_name1 {  
    <type> <identifier_list>;  
    <type> <identifier_list>;  
    ...  
};
```

```
struct struct_name2 {  
    struct struct_name1;  
    <type> <identifier_list>;  
    ...  
};
```

Ex: Nested structure



- Write a program to find the distance of a line.

Ex: Nested structure

- Write a program to find the distance of a line.

Steps

- Create the nested structure - Point appearing as a member of Line

Ex: Nested structure

- Write a program to find the distance of a line.

Steps

- Create the nested structure – Point appearing as a member of Line
- Function to compute the distance between two points

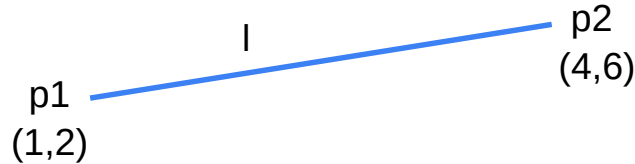
$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Ex: Nested structure

- Write a program to find the length of a line.

Steps

- Create the nested structure – Point appearing as a member of Line
- Function to compute the distance between two points
- Initialize the line and call the function from main



Ex: Nested structure



- Write a program to store student records
 - Attributes of student include name and rollno
 - Each student enrolls for 4 courses
 - Attributes of course include course code, course name and marks
 - You need to record the marks scored by 60 students and report their average mark.

Ex: Nested structure

- Write a program to store student records
 - Attributes of student include name and rollno
 - Each student enrolls for 4 courses
 - Attributes of course include course code, course name and marks
 - You need to record the marks scored by 60 students and report their average mark.

Steps:

- Create the nested structure – course appearing as a member of student

```
typedef struct {  
    char c_code[5];  
    char c_name[20];  
    float marks;  
} course;  
  
typedef struct {  
    char s_name[30];  
    int roll_no;  
    course subjects[4];  
} student;  
  
student bsc_student[250];
```

Ex: Nested structure

- Write a program to store student records
 - Attributes of student include name and rollno
 - Each student enrolls for 4 courses
 - Attributes of course include course code, course name and marks
 - You need to record the marks scored by 60 students and report their average mark.

Steps:

- Create the nested structure – course appearing as a member of student
- Read input

```
for(int i = 0; i < 2; i++) {  
    scanf("%s", bsc_student[i].s_name);  
    scanf("%d", &bsc_student[i].roll_no);  
    for (int j = 0; j < 4; j++) {  
        scanf("%s",  
bsc_student[i].subjects[j].c_code);  
        scanf("%s",  
bsc_student[i].subjects[j].c_name);  
        scanf("%f",  
&bsc_student[i].subjects[j].marks);  
    }  
}  
}
```

Assignment



- Assign whole structures using =
- Consider the following declaration

```
typedef struct {  
    char name[30];  
    int roll_no;
```

```
} student;
```

```
student stud1 = {"Alice", 123}, stud2;
```

```
stud2 = stud1;
```

```
printf("%s\n", stud2.name);
```

Output : Alice





User-defined data type

Function and structures



struct as function return type

- struct can be used as a function return type

```
struct student {  
    char name[30];  
    int roll_no;  
};
```

- Write a function to create a new student.
 - The function will take name and roll number as arguments
 - Create a structure student
 - Return the student

struct as Parameters

- A single struct is passed by value.
 - all of its components are copied from the argument (actual parameter) to initialize the (formal) parameter.

Putting it all together



```
#include<stdio.h>
#include<string.h>

struct student{
    char name[10];
    int roll_no;
};

struct student createStudent(const char *name, int
roll_no){
    struct student newStudent;
    strncpy(newStudent.name,name,sizeof(newStudent
.name));
    newStudent.roll_no = roll_no;
    return newStudent;
}
```

```
void printStudent(struct student s){
    printf("Name %s\n", s.name);
    printf("Roll no %d\n", s.roll_no);
}

int main(){
    struct student stud1 =
createStudent("Alice", 1234);
printStudent(stud1); }
```





User-defined data type

Pointers to structures



Pointers and structures

- Consider the declaration

```
struct student {  
    char name[30];  
    int roll_no;  
    float marks;  
} msc_students[60], *ptr;
```

- Reference ptr to the structure array
ptr = msc_students;



Accessing members of structures

- Members of a “structure pointer” can be accessed in two ways
 - `(*ptr).field;` // Member operator “.” has higher precedence than “*”
 - `ptr->field;` // Less confusing
 - The symbol “->” is called the arrow operator.
- The operator “->” enjoys highest priority among operators.



Ex: Pointer and structures

- Write a structure to store student record.
 - Read input and print output using pointers to the structure

Ex: Pointer and structures

- Write a structure to store student records.
 - Read input and print output using pointers to the structure
- Pointer to structure

```
struct student {  
    char name[30];  
    int roll_no;  
    float marks;  
} msc_students[60], *ptr;  
  
ptr = msc_students;
```

Review



- Use pointers to access elements of the array of structures

```
struct student {  
    char name[30];  
    int roll_no;  
    float marks;  
} msc_students[60], *ptr = msc_student;
```

```
for (int i = 0; i < 60; i++){  
    scanf("%s", ptr->name);  
    scanf("%d",&ptr->roll_no);  
    scanf("%f",&(*ptr).marks);  
    ptr++;  
}
```

```
ptr = msc_student;  
for (int i = 0; i < 60; i++){  
    printf("%s\t", ptr->name);  
    printf("%d\t",ptr->roll_no);  
    printf("%f\n",(*ptr).marks);  
    ptr++;  
}
```

Review



- This also works!

```
for (int i = 0; i < 60; i++){  
    printf("%s\t", ptr[i].name);  
    printf("%d\t", ptr[i].roll_no);  
    printf("%f\n", ptr[i].marks);  
}
```

Declarations and Initializations

```
struct student {
    char *last_name;
    int student_id;
    char grade; } ;
struct student tmp, *p = &tmp;
tmp.grade = 'A'; tmp.last_name = "Casanov";
tmp.student_id = 910017;
```

Expression	Equivalent Expression
tmp.grade	$p \rightarrow grade$ 'A'
tmp.last_name	$p \rightarrow last_name$ 'C'
(*p).student_id	$p \rightarrow student_id$ 910017
* p->last_name + 1	$p \rightarrow last_name[0] + 1$ 'D'
*(p->last_name +2)	$p \rightarrow last_name[2]$ 'S'



User-defined data type

Dynamic memory allocation to structures



Exercise



- Write a C program to find the distance from a point to $n-1$ other points.
 - Read an integer n as user input
 - Read n points
 - From the first point find the distance to $n-1$ other points

Dynamic memory allocation to structures



1. Declare the structure - Point
2. Read n from the user



Dynamic memory allocation to structures

1. Declare the structure - Point
2. Read n from the user
3. Allocate memory for the n points
 - Use `calloc()`

Dynamic memory allocation to structures

1. Declare the structure - Point
2. Read n from the user
3. Allocate memory for the n points
4. Read the n points

Dynamic memory allocation to structures

1. Declare the structure - Point
2. Read n from the user
3. Allocate memory for the n points
4. Read the n points
5. Calculate distance of $n-1$ points from the first point
6. Free the memory allocated



Other user-defined data type

Enumeration



Enumeration syntax

- It is a user-defined data type
- Use the key word enum
- Syntax is similar to that of structure

```
enum enum_name {  
    enumerator1,  
    enumerator2,  
    // ...  
    enumeratorN  
};
```

Ex: Enumeration

- The enumerators are constants of type int
- Example:

```
enum Days {  
    Sunday,    // 0  
    Monday,    // 1  
    Tuesday,   // 2  
    Wednesday, // 3  
    Thursday,  // 4  
    Friday,    // 5  
    Saturday   // 6  
};
```

```
enum Days d1, d2;
```

Enumeration



- It is a user-defined data type
- Allows the programmer to define a set of named integral constants.
- These constants are called enumerators
- Provide a way to assign clear, meaningful names to specific integer values, enhancing code readability and maintainability.



Ex: Enumeration

- Initialize and use enum values:

```
enum Days {  
    Sunday,    // 0  
    Monday,    // 1  
    Tuesday,   // 2  
    Wednesday, // 3  
    Thursday,  // 4  
    Friday,    // 5  
    Saturday   // 6  
};
```

```
enum Days d1, d2;
```

```
d1 = Wednesday;
```

```
printf("%d",d1);
```

Enumeration: Use cases



- Makes code more self explanatory

```
enum Days day = Sunday;
```

```
switch (day) {  
    case Sunday:  
        printf("Sunday");  
        break;  
    case Monday:  
        printf("Monday");  
        break;  
    .....  
}
```



Ex:



- Write a program to return the number of days in a month using enum.



Ex:



- Write a program to return the number of days in a month using enum.

Steps

1. Declare an enumeration for months

```
enum Months {  
    January, // 0  
    February, // 1  
    March, // 2  
    April, // 3  
    May, // 4  
    June, // 5  
    July, // 6  
    August, // 7  
    September, // 8  
    October, // 9  
    November, // 10  
    December // 11  
};
```



Ex:



- Write a program to return the number of days in a month using enum.

Steps

1. Declare an enumeration for months
2. Take user input
3. Check for invalid input

```
int main() {  
    int month;  
    printf("Enter the month (as a number, 0-11): ");  
    scanf("%d", &month);  
    if (month < January || month > December) {  
        perror("Invalid month number.\n");  
        exit(1);  
    }  
}
```



Ex:



- Write a program to return the number of days in a month using enum.

Steps

1. Declare an enumeration for months
2. Take user input
3. Check for invalid input
4. Call function – returns the number of days in a month
 - Write the function



```
int numDays = DaysInMonth(month);
printf("Number of days in the selected month: %d\n", numDays);
}
int DaysInMonth(enum Months month) {
    switch (month) {
        case April:
        case June:
        case September:
        case November:
            return 30;
        case February:
            return 28; // Assuming it's not a leap year
        default:
            return 31;
    }
}
```

Ex: Use the enumerator

- Using enumerator to make the program more readable

```
int main() {  
    int month;  
    printf("Enter the month (as a number, 0-11): ");  
    scanf("%d", &month);  
    if (month < January || month > December) {  
        perror("Invalid month number.\n");  
        exit(1);  
    }  
    int numDays = DaysInMonth(month);  
    printf("Number of days in the selected month: %d\n", numDays);  
    return 0;  
}
```